ISE 3434: Applied Optimization

Assignment #2

Due Date: October 2, 2026, 11:59pm

Directions

- Submit one PDF file (your write-up) and one Jupyter notebook (.ipynb) file to Canvas by the due date above. Late submissions are accepted according to the late policy in the syllabus.
- For Problems 1 to 3, your PDF should clearly define the model (variables, objective, constraints, and nonnegativity & integer restrictions), and briefly describe your solution along with the objective value.
- Your notebook should contain working code that prints your results.
- **The data for Problems 1 to 3 is provided as CSV files on Canvas** along with this assignment file. Load them with pandas, as shown in Coding Session 2, rather than typing the data by hand.
- If a model does not solve to optimality within its time limit, report the best solution found and the best bound instead of an optimal value.
- Grading considers both the correctness and clarity of your written solutions, and the correctness and readability of your code.

Problem 1: Cell Tower Placement (25 points)

A city is planning where to place cell towers so that every residential area has mobile network coverage. The city is divided into 25 regions, labeled 1 through 25. A region is covered if it contains a cell tower or is adjacent to a region that contains a cell tower. Because of budget constraints, the city wants to install as few towers as possible while covering every region.

The adjacency relationships between regions are provided in **city_adjacency.csv** as a 25 by 25 matrix. The (i, j) entry equals 1 if regions i and j are adjacent, and 0 otherwise. Load this matrix from the file and use it to build your constraints.

Model the problem as a binary integer program and solve it using Gurobi.

To submit

- Model description: variables, objective, constraints, and nonnegativity & integer restrictions.
- Your code as part of the single .ipynb file, with results printed.
- A brief written summary of your solution, including the regions where towers are installed and the optimal objective value.

Problem 2: Storage Optimization for Moving Boxes, Revisited (30 points)

You are again helping a friend move, as in Problem 3 of Assignment 1. This time, however, your friend has many items of each type: there are 15 item types, and your friend owns several items of every type. Each moving box holds up to 50 cubic feet, and the moving company charges $30 for each box used. The goal is to pack every item while using as few boxes as possible, without exceeding the capacity of any box.

The volume of one item of each type (in cubic feet) and the number of items of that type are given below.

Type	Volume	Quantity	Type	Volume	Quantity
1	12	144	9	9	152
2	7	33	10	14	79
3	19	196	11	11	54
4	10	30	12	18	187
5	24	111	13	16	84
6	8	25	14	13	52
7	15	104	15	17	21
8	22	103			

This data is provided in **items_data.csv**.

Note 1. Think carefully about how this model differs from Problem 3 of Assignment 1. There, your friend had one item of each type. Here, your friend has many items of each type.

Note 2. Before building the model, you need an upper bound on the number of boxes required, in order to define the set of available boxes. In Assignment 1 this bound was given to you (8 boxes). Explain in your write-up how you obtained your bound. If you are unable to determine one, use 1000.

Note 3. Depending on your model, the code may take a long time to run. Set a time limit of 10 minutes, and report the number of boxes used in the best solution found and the best bound found within that time.

Model the problem as a mixed integer program and solve it using Gurobi.

To submit

- Model description: variables, objective, constraints, and nonnegativity & integer restrictions.
- Your code as part of the single .ipynb file, with results printed.
- A brief written summary of your solution, including the number of boxes used in the best solution found, and the best bound found within 10 minutes.

Problem 3: College Football Tournament Scheduling (30 points)

You are organizing a college football tournament among 50 university teams located across the United States. Each team plays exactly one match, against exactly one other team, and for each match one of the two teams travels to the other team's city. The goal is to pair the teams so that the total travel distance over all matches is as small as possible.

The file **tournament_teams.csv** lists each university along with the latitude and longitude of its city. Compute the travel distance between every pair of cities using the haversine formula from Coding Session 2, and plot your pairings on a map using folium.

Model the problem as a binary integer program and solve it using Gurobi.

To submit

- Model description: variables, objective, constraints, and nonnegativity & integer restrictions.
- Your code as part of the single .ipynb file, with results printed.
- A brief written summary of your solution, including the minimum total travel distance in miles, the time taken to solve the model, and a map of the pairings.

Problem 4: Heuristics for the TSP and Vehicle Routing (15 points)

Watch the video *The Traveling Salesman Problem: When Good Enough Beats Perfect* at https://www.youtube.com/watch?v=GiDsjIBOVoA. It explains why solving the TSP to optimality becomes impractical for large instances, and describes several heuristics that find good tours quickly. Then answer the following questions.

(a) Mention something that you learned from the video.

(b) In your own words, describe ant colony optimization.

(c) The Vehicle Routing Problem (VRP) extends the TSP to several vehicles. All vehicles start and end at a common depot, every customer must be visited by exactly one vehicle, and each vehicle can visit only a limited number of customers. The goal is to minimize the total distance driven by all vehicles. For example, a delivery company with one warehouse and 3 trucks, each able to visit at most 5 customers, must serve 15 customers.

Choose one of the TSP heuristics from the video, and describe in a few sentences how you would adapt it to build a feasible solution to this VRP. Explain why you expect your approach to give a reasonably good solution, and describe one drawback of it.

To submit

- Your answers to parts (a), (b), and (c) in the PDF. No model or code is required for this problem.